Lecture 4

Logical gates model elementary computational steps in digital electronic circuits. For example:

x	NOT(x)
0	1
1	0

xy	AND(x,y)	OR(x,y)	XOR(x,y)
00	0	0	0
01	0	1	1
10	0	1	1
11	1	1	0

Algebraically (modulo 2): $\mathrm{NOT}(x) = x+1$, $\mathrm{AND}(x) = x \cdot y$, $\mathrm{XOR}(x,y) = x+y$.

Any Boolean function $f: \{0,1\}^n \to \{0,1\}$ can be expressed using just AND, OR, NOT. For example, equality:

xy	EQ(x,y)
00	1
01	0
10	0
11	1

$\mathrm{EQ}(x,y) = 1$

$\Updownarrow$

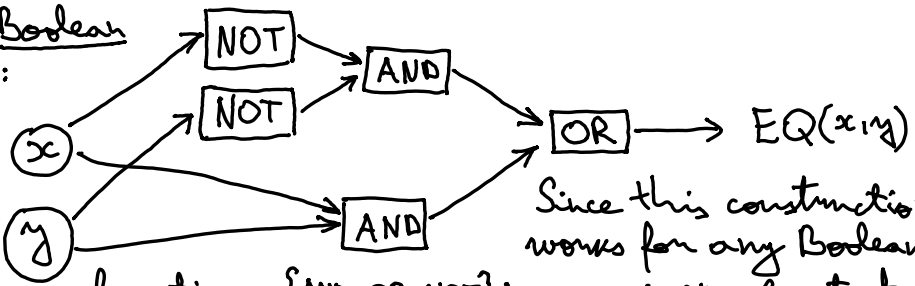
$x = y$

$\Updownarrow$

$(x=0 \text{ and } y=0)$ or $(x=1 \text{ and } y=1)$

This gives the formula: $\mathrm{EQ}(x,y) = \mathrm{OR}(\mathrm{AND}(\mathrm{NOT}(x), \mathrm{NOT}(y)), \mathrm{AND}(x,y))$

On the Boolean circuit:



Since this construction works for any Boolean function, {AND, OR, NOT} is a universal set of logical gates. In fact, just {AND, NOT} or even just {NAND} would also do the job.

Single-qubit gates

We saw in Lecture 1 that a general one-qubit operation is of the form $U = \begin{pmatrix} a & c \\ b & d \end{pmatrix}$ where $a, b, c, d \in \mathbb{C}$ are such that $|a|^2 + |b|^2 = 1$, $|c|^2 + |d|^2 = 1$, $a\bar{c} + b\bar{d} = 0$. This is equivalent to $U^\dagger U = I$, meaning U is unitary.

Thus, a <u>one-qubit gate</u> is any 2×2 unitary.

Expanded in the computational basis:

$$U = \sum_{i,j=0}^{1} U_{i,j} |i\rangle\langle j|$$

time ←

$$|\psi_{out}\rangle = VU|\psi_{in}\rangle$$

Graphical notation:



time →



<u>Example</u> The <u>Hadamard gate</u> $H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ changes basis between the computational basis $\{|0\rangle, |1\rangle\}$ and the Hadamard basis $\{|+\rangle, |-\rangle\}$:

$$H|0\rangle = |+\rangle \qquad H|+\rangle = |0\rangle$$
$$H|1\rangle = |-\rangle \qquad H|-\rangle = |1\rangle$$



<u>Example</u> The <u>Pauli gates</u>:



$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

You can check that $X^2 = Y^2 = Z^2 = I$ and $XY = -YX = iZ$.

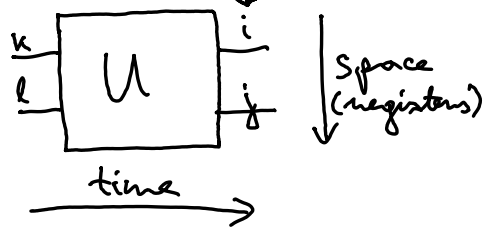
<u>Note</u>: They are closely related to quaternions.

Multi-qubit quantum gates

A two-qubit gate is a 4×4 unitary matrix:

$$U=\sum_{i,j,k,l=0}^{1} U_{ij,kl}\,|i,j\rangle\langle k,l| = \sum_{i,j,k,l=0}^{1} U_{ij,kl}\,|i\rangle\langle k|\otimes|j\rangle\langle l|$$

output, input, time



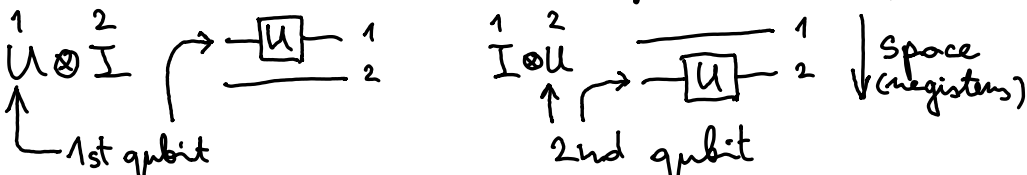
Example: $|0\rangle\langle0|\otimes|0\rangle\langle0| + |0\rangle\langle1|\otimes|0\rangle\langle1| + |1\rangle\langle0|\otimes|1\rangle\langle0| + |1\rangle\langle1|\otimes|1\rangle\langle1|$ is the 2-qubit identity operation.

$$\begin{pmatrix} 1 & & & \\ & 1 & & \\ & & 1 & \\ & & & 1 \end{pmatrix} \begin{matrix} |00\rangle \\ |01\rangle \\ |10\rangle \\ |11\rangle \end{matrix}$$

Similarly, an n-qubit gate is a $2^n\times2^n$ unitary. It has n input and n output qubits.

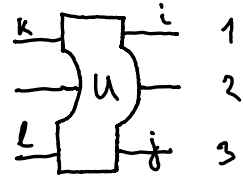
Local operations

If a 1-qubit gate is applied locally on one of two qubits, we can write it as a 2-qubit unitary:



If a 2-qubit gate is applied locally on qubits 1 and 3, we can write this as a 3-qubit operation:

$$\sum_{i,j,k,l=0}^{1} U_{ij,kl}\,|i\rangle\langle k|\otimes I\otimes|j\rangle\langle l|$$

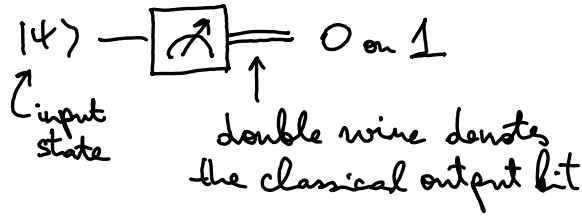


Single- and multi-qubit measurements

The standard basis measurement of a qubit is denoted by

$|\psi\rangle = \alpha_0|0\rangle + \alpha_1|1\rangle$

$|\alpha_0|^2$ → 0, $|\alpha_1|^2$ → 1



We can describe such measurement using the operators (row vectors) $\{\langle 0|, \langle 1|\}$. Indeed, the outcome $i \in \{0,1\}$ occurs with probability $p_i = |\langle i|\psi\rangle|^2$. If you want to also keep the post-measurement state, you can instead use the operators (2×2 matrices) $\{|0\rangle\langle 0|, |1\rangle\langle 1|\}$. Let $\Pi_i = |i\rangle\langle i|$. Then

$$p_i = \|\Pi_i|\psi\rangle\|^2 = \||i\rangle\langle i|\psi\rangle\|^2 = \underbrace{\||i\rangle\|^2}_{1} \cdot |\langle i|\psi\rangle|^2$$

and the state after the measurement is

$$|\psi_i\rangle = \frac{\Pi_i|\psi\rangle}{\sqrt{p_i}} = \frac{|i\rangle\langle i|\psi\rangle}{|\langle i|\psi\rangle|} \sim |i\rangle.$$

(∼ : equal up to a global phase)

This is called projective measurement since Π_i are projectors. The same formulas work for any orthonormal basis.

If you measure only one out of two qubits, you can tensor the measurement operators with I:

$\{\langle i|\otimes I\}$ $|\psi\rangle$ → $|\psi_i\rangle$ $\quad$ $\{I\otimes\langle i|\}$ $|\psi\rangle$ → $|\psi_i\rangle$

$p_i = \|(\langle i|\otimes I)|\psi\rangle\|^2$

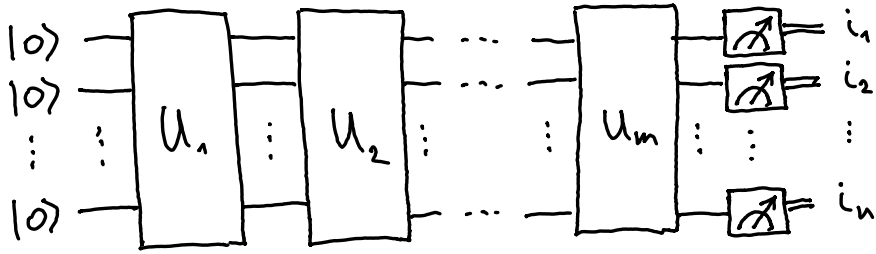
$|\psi_i\rangle = \frac{(\langle i|\otimes I)|\psi\rangle}{\sqrt{p_i}}$ ← remaining state on the 2nd qubit

$p_i = \|(I\otimes\langle i|)|\psi\rangle\|^2$

$|\psi_i\rangle = \frac{(I\otimes\langle i|)|\psi\rangle}{\sqrt{p_i}}$

Quantum circuits

An n-qubit quantum circuit is a sequence of n-qubit unitary operations, followed by the measurement in the standard basis:



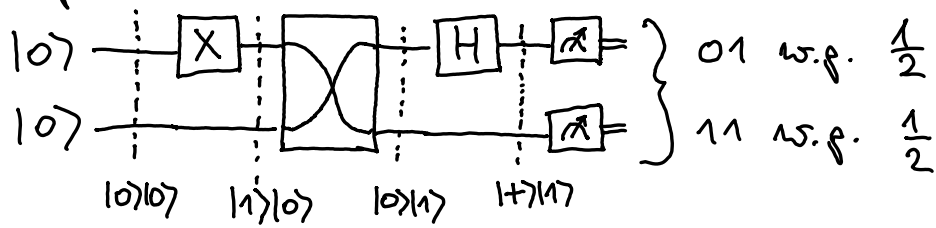
The output is an n-bit string $i_1 i_2 \dots i_n \in \{0,1\}^n$, with probability

$$\left| \langle i_1 i_2 \dots i_n | \underbrace{U_m \dots U_2 U_1}_{\text{computation}} \underbrace{|00\dots0\rangle}_{|0\rangle^{\otimes n}} \right|^2 .$$

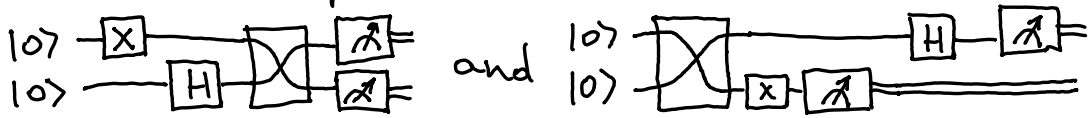
output ↑ computation ↑ input

Example

SWAP $|i\rangle \otimes |j\rangle = |j\rangle \otimes |i\rangle$



This circuit is equivalent to



Fact: Any intermediate measurement can be deferred to the end. Measurements in any other orthonormal basis can always be converted to the standard basis measurement.

Locality and uniformity

Computation is local if it consists of elementary operations that act only on a few qubits at a time.

Example (for bits): Any Boolean function $f: \{0,1\}^n \to \{0,1\}$ can be computed using logical gates (e.g., AND and NOT) with at most two input bits.

We would like quantum circuits to be local as well because:
- it is hard to make more than two quantum systems to interact simultaneously,
- the hardware might have only a finite number of different types of interactions available (in particular, they act only on some finite number of qubits),
- we cannot afford to store $2^n \times 2^n$ matrices to describe the algorithm (e.g., when $n=50$, there are not enough atoms in the universe to write down each matrix entry, even if we use one atom per entry).

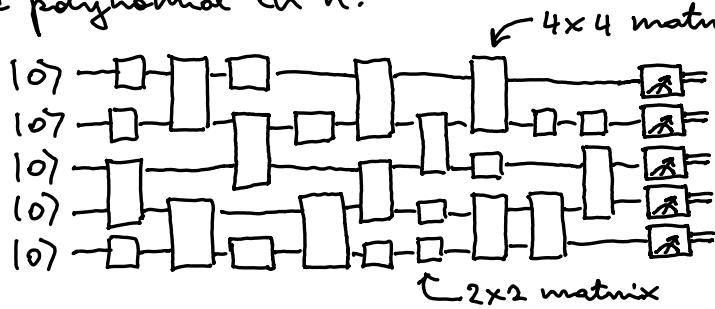
To solve a computational problem, we need not just a single circuit, but a family of circuits – one circuit for every input size of the problem (e.g., we cannot simply use the circuit that adds 3-bit numbers to add 5-bit numbers). But where do we get these circuits in the first place? They are generated by some other (classical) algorithm.

A uniform family of circuits is one that can be produced by a deterministic Turing machine in polynomial time.

This excludes the possibility of generating an (exponential size) look-up table that stores all possible answers.

Quantum algorithm

A quantum algorithm is an infinite family $C_1, C_2, \ldots$ of quantum circuits, where C_n acts on n qubits and consists of a finite sequence of 1-qubit and 2-qubit gates: $C_n = (U_1, U_2, \ldots, U_{L(n)})$, where $L(n)$ denotes the number of gates in the circuit. The map $n \mapsto C_n$ must be efficiently computable (in polynomial time on a deterministic Turing machine). In particular, $L(n)$ must be polynomial in n.



This is how a typical quantum algorithm might look like. All gates act on at most two (neighbouring) qubits and are easy to specify since they involve at most $4 \times 4 = 16$ complex numbers. To fully specify the algorithm, one must provide at most $16 \cdot L(n)$ complex numbers, which is only polynomial in n, plus at most two $\lceil \log_2 n \rceil$-bit numbers per gate to specify on which qubits it acts on. The total amount of information is still polynomial in n and hence in principle manageable.

In contrast, the overall unitary transformation implemented by the algorithm is described by a $2^n \times 2^n$ matrix, which is too big to even write down. This also implies that not all unitary n-qubit transformations can be implemented by a poly(n)-size quantum circuit.